Laporan Tugas Besar 2

Implementasi Algoritma Pembelajaran Mesin

Disusun untuk memenuhi tugas mata kuliah IF3170 Inteligensi Artifisial pada Semester 1 (Ganjil) Tahun Akademik 2025/2026

Kelompok 21 (`learning_rate`)

Razi Rachman Widyadhana	13523004
Adhimas Aryo Bimo	13523052
Muhammad Fathur Rizky	13523105
Guntara Hambali	13523114
Ahmad Wicaksono	13523121

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Bab I: Deskripsi Persoalan **1**

Bab II: Landasan Teori **2**
2.1. Supervised Learning 2
2.2. Decision Tree Learning 2
2.3. Logistic Regression 3
2.4. Support Vector Classifier (SVC) 4
2.5. F1-Score Macro 5

Bab III:Tahap Pengolahan dan Persiapan Data **6**
3.1. Diagram Alur 6
3.2. Pembagian Fitur Berdasarkan Jenisnya 7
3.3. *Z-score clipping* 8
3.4. Standard Scaler 8
3.5. One Hot Encoding 10
3.6. Label Encoding 12
3.7. Preprocessing Pipeline 13
3.8. Feature Selection 15

Bab IV: Desain Pemodelan **21**
4.1. Cross Validation 21
4.2. Metric 24
4.3. Implementasi Algoritma Model 25
4.4. Implementasi Bonus 33

Bab V: Hasil dan Analisis **37**
5.1. Exploratory Data Analysis 37
5.2. Pelatihan dan Validasi Model 39
5.3. Analisis dan Pembahasan 44

Bab VI: Penutup **45**
6.1. Kesimpulan 45
6.2. Saran 45

Lampiran **46**

Referensi **47**

Bab I
Deskripsi Persoalan

Latar Belakang

Dalam konteks pendidikan tinggi, data mahasiswa dapat dimanfaatkan untuk membantu institusi memahami faktor yang berpotensi mempengaruhi keberhasilan akademik. Dengan adanya data demografis, informasi sosial-ekonomi, serta performa akademik pada semester awal, *machine learning* dapat digunakan untuk memprediksi apakah seorang mahasiswa berpotensi lulus tepat waktu, mengalami penurunan performa, atau bahkan berisiko berhenti kuliah.

Salah satu metode utama yang dapat menjadi solusi adalah penerapan algoritma pembelajaran mesin seperti *Decision Tree Learning*, *Logistic Regression*, dan *Support Vector Machine*, yang dirancang untuk membangun model prediktif secara iteratif dengan memaksimalkan akurasi klasifikasi berdasarkan fitur-fitur relevan seperti demografi dan capaian akademik. Pendekatan ini memungkinkan institusi pendidikan untuk menganalisis pola data secara otomatis dan menghasilkan prediksi yang lebih akurat serta mendukung intervensi dini bagi mahasiswa berisiko.

Dataset yang digunakan pada tugas besar ini berisi beragam atribut mahasiswa yang terdaftar di program sarjana, mulai dari status pendaftaran, nilai masuk, pekerjaan orang tua, hingga capaian akademik semester 1 dan 2. Informasi tersebut dapat diolah menjadi model prediktif yang membantu institusi mengambil keputusan berbasis data.

Rumusan Masalah

Berdasarkan latar belakang tersebut, permasalahan yang diangkat dalam tugas besar ini dapat dirumuskan sebagai berikut:

1. Bagaimana rancangan *pipeline* dalam pemrosesan data mahasiswa agar siap untuk model *machine learning*?
2. Bagaimana mengimplementasikan model pembelajaran mesin *Decision Tree Learning* (DTL), *Logistic Regression*, dan *Support Vector Machine* (SVM)?
3. Model mana yang memberikan performa prediksi paling baik berdasarkan metrik evaluasi, terutama jika dataset memiliki kelas target yang tidak seimbang?

Tujuan

Sejalan dengan rumusan masalah, tujuan dari tugas besar ini adalah sebagai berikut:

1. Merancang *pipeline* pemrosesan data melalui *data cleaning*, *transformation*, dan *feature engineering* agar terotomatisasi dan *reproducible*.
2. Mengimplementasikan model pembelajaran mesin DTL, *Logistic Regression*, dan SVM secara manual tanpa *library* model bawaan.
3. Melatih, mengevaluasi, dan membandingkan performa model (manual vs. *scikit-learn*) menggunakan metrik relevan serta menganalisis insight dari hasil perbandingan.

Bab II
Landasan Teori

2.1. Supervised Learning

Supervised Learning adalah paradigma pembelajaran mesin di mana agen belajar dari sekumpulan data yang sudah diberi label dengan benar. Setiap data poin dalam set pelatihan terdiri dari **pasangan *input-output***.

- **Input:** Disebut juga fitur, atribut, atau variabel independen (disimbolkan dengan X).
- **Output:** Disebut juga label, target, atau variabel dependen (disimbolkan dengan y atau $f(x)$).

FITUR/ATRIBUT | LABEL/KELAS

Jenis Buah	Harga	Jenis Penjualan	Laku Terjual
Apel	Rp. 10.000	10	Ya
Jeruk	Rp. 7.500	15	Ya
Pisang	Rp. 5.000	8	Tidak

Gambar 1: Input dan Output dalam Supervised Learning

Supervised Learning umumnya dibagi menjadi dua jenis tugas utama, tergantung pada domain dari label *output*-nya:

1. **Klasifikasi (Classification):**

   Tujuannya adalah memprediksi label kategori/diskrit. Domain dari labelnya adalah sebuah himpunan nilai yang terbatas. Contoh: Memprediksi apakah sebuah email adalah 'spam' atau 'bukan spam'.

2. **Regresi (Regression):**

   Tujuannya adalah memprediksi nilai numerik/kontinu. Domain dari labelnya bisa berupa nilai riil apa pun. Contoh: Memprediksi harga rumah berdasarkan luasnya; memprediksi suhu besok hari.

2.2. Decision Tree Learning

Decision Tree Classifier merupakan metode klasifikasi berbasis pemisahan rekursif terhadap ruang fitur untuk memaksimalkan homogenitas kelas pada setiap node. Model ini memilih atribut terbaik pada setiap tahap pemisahan dengan mengukur impurity suatu node menggunakan metrik seperti **Entropy** dan **Gini Impurity**. Entropy digunakan untuk mengukur ketidakpastian distribusi kelas dalam node dan dirumuskan sebagai:

$$H(S) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

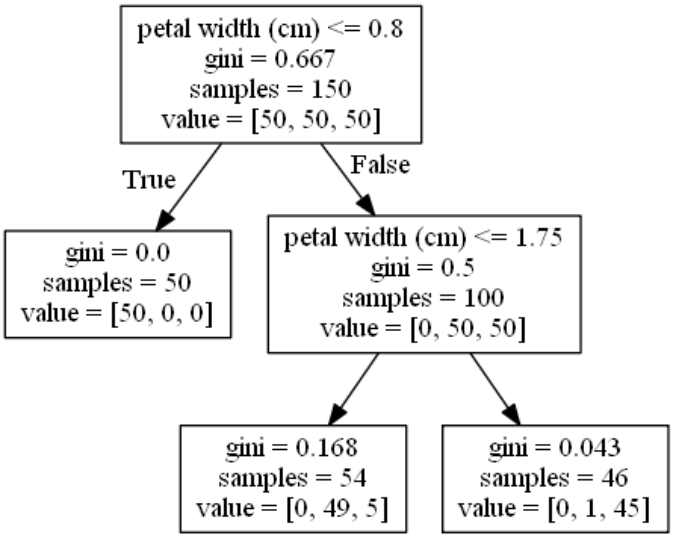
dalam hal ini p_k adalah proporsi sampel dalam kelas (k). Sementara itu, Gini Impurity menghitung probabilitas kesalahan klasifikasi apabila suatu sampel dipilih secara acak dan diberi label secara acak berdasarkan distribusi kelas node tersebut.

$$G(S) = 1 - \sum_{k\,=\,1} p_k^2$$

Penurunan impurity akibat split dihitung melalui Information Gain, yaitu:

$$IG(S, A) = H(S) - \sum_{v\,\in\,A} \frac{|S_v|}{|S|} H(S_v)$$

Dengan memilih atribut yang memaksimalkan (IG), pohon secara bertahap membangun struktur yang optimal secara lokal untuk pemisahan kelas. Decision Tree banyak digunakan karena sifat interpretabilitasnya yang tinggi serta fleksibilitas dalam menghadapi fitur numerik maupun kategorikal [1].



Gambar 2: Ilustrasi Model Decision Tree dengan metrik Gini Impurity

2.3. Logistic Regression

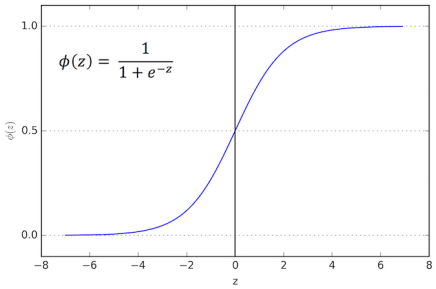
Logistic Regression adalah model klasifikasi biner yang memodelkan probabilitas suatu sampel berada dalam kelas positif menggunakan fungsi logistik. Hubungan antara fitur (x) dan probabilitas kelas positif $p(y = 1|x)$ diberikan oleh fungsi sigmoid:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w^T x + b$$

Model ini dilatih dengan meminimalkan *Binary Cross-Entropy Loss*, yang secara formal dituliskan sebagai:

$$\mathcal{L}(w, b) = -\sum_{i\,=\,1}^{n} [y_i \ \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i))]$$

Optimisasi biasanya dilakukan menggunakan metode gradien seperti Gradient Descent atau Quasi-Newton (misalnya L-BFGS). Logistic Regression banyak diaplikasikan pada domain yang membutuhkan interpretabilitas parameter, serta efektif sebagai baseline untuk masalah klasifikasi dengan batas keputusan yang mendekati linear [2].



Gambar 3: Ilustrasi Model Logistic Regression

2.4. Support Vector Classifier (SVC)

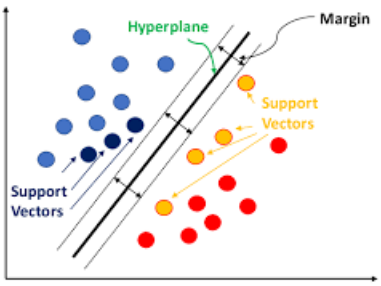
Support Vector Classifier merupakan metode klasifikasi yang mencari hyperplane optimal dengan **margin maksimum** untuk memisahkan dua kelas. Masalah optimisasi pada SVC untuk kasus linear separable dirumuskan sebagai:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1, \quad \forall i$$

dengan (w) sebagai vektor bobot dan (b) sebagai bias. Untuk data yang tidak sepenuhnya separable, digunakan soft-margin SVM, yaitu:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

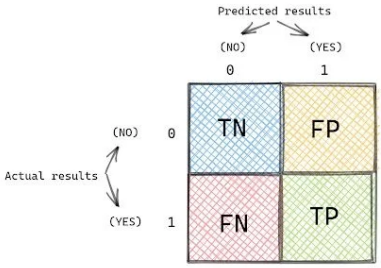
Untuk menangani data non-linear, SVC menerapkan ***kernel trick*** sehingga perhitungan dot product dilakukan dalam ruang fitur berdimensi tinggi tanpa menghitung transformasinya secara eksplisit. Kernel yang umum digunakan meliputi Radial Basis Function (RBF), polynomial, dan sigmoid. SVC dikenal memiliki performa yang sangat baik terutama pada dataset berdimensi tinggi atau dataset dengan batas keputusan kompleks [3].



Gambar 4: Ilustrasi Model Support Vector Machine yang *linearly seperable*

2.5. F1-Score Macro

F1-Score Macro merupakan salah satu metrik evaluasi yang digunakan untuk menilai performa model klasifikasi terutama pada kondisi data yang tidak seimbang (imbalanced). Metrik ini menghitung F1-score secara independen untuk setiap kelas, kemudian mengambil rata-rata aritmetika dari seluruh nilai F1 tersebut, sehingga setiap kelas memiliki kontribusi yang sama meskipun distribusinya tidak seimbang. Untuk memudahkan, diberikan terlebih dahulu *confusion matrix*



Gambar 5: Contoh *confusion matrix*

di mana (TP_k), (FP_k), dan (FN_k) masing-masing adalah jumlah true positive, false positive, dan false negative pada kelas (k). Untuk setiap kelas (k), *Precision* dan *Recall* didefinisikan sebagai:

$$Precision_k = \frac{TP_k}{TP_k + FP_k}, \quad Recall_k = \frac{TP_k}{TP_k + FN_k},$$

F1-score untuk kelas (k) merupakan harmonic mean antara Precision dan Recall:

$$F1_k = 2 \cdot \frac{Precision_k \cdot Recall_k}{Precision_k + Recall_k}.$$

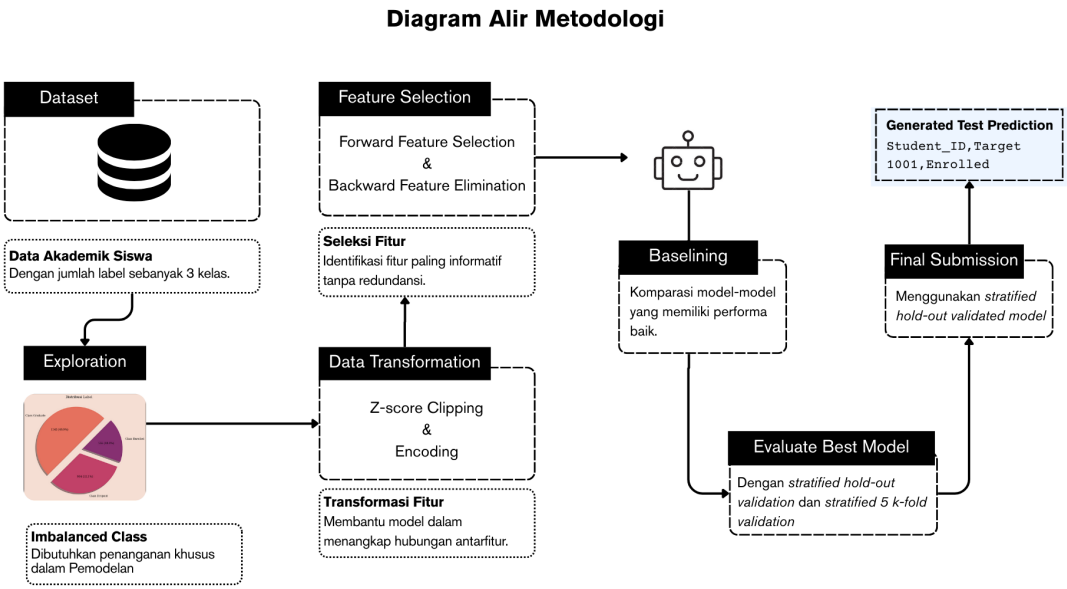
Nilai Macro F1-score diperoleh dengan menghitung rata-rata dari seluruh F1-score kelas:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} F1_k.$$

Berbeda dengan Micro F1 yang menimbang kelas berdasarkan jumlah sampel, Macro F1 memberikan bobot yang sama kepada seluruh kelas, menjadikannya metrik yang lebih representatif dalam mengukur performa model pada dataset dengan distribusi kelas yang tidak seimbang [4].

Bab III
Tahap Pengolahan dan Persiapan Data

3.1. Diagram Alur



Gambar 6: Diagram Alur Eksperimen

Diagram alur eksperimen dimulai dari dataset akademik siswa yang memiliki target berupa tiga kelas performa akademik. Tahap pertama adalah eksplorasi data untuk memahami distribusi kelas dan karakteristik awal dataset. Dari hasil eksplorasi terlihat adanya ketidakseimbangan kelas (imbalanced class) yang cukup signifikan, sehingga pada tahap pemodelan nantinya diperlukan penanganan khusus agar model tidak bias terhadap kelas mayoritas.

Selanjutnya, data mengalami beberapa proses preprocessing secara berurutan. Pertama dilakukan transformasi data dengan Z-score clipping untuk menangani outlier dan encoding untuk mengubah data kategorikal menjadi numerik. Setelah itu dilakukan seleksi fitur menggunakan kombinasi Forward Feature Selection dan Backward Feature Elimination untuk mendapatkan fitur-fitur paling informatif tanpa redundansi. Proses ini membantu meningkatkan kualitas input yang akan digunakan oleh model.

Tahap terakhir adalah pemodelan dan evaluasi. Berbagai model manual dan sklearn dicoba dan dibandingkan performanya. Model terbaik kemudian dievaluasi secara ketat menggunakan *stratified hold-out validation* dan *stratified 5-fold cross validation* untuk memastikan hasil yang stabil meskipun data tidak seimbang. Model manual terbaik sebagai model final yang terpilih kemudian digunakan untuk memprediksi data test (Student_ID 1001 dan seterusnya berlabel "Enrolled") dan menghasilkan file submission akhir. Seluruh alur dirancang agar tetap menjaga stratifikasi kelas pada setiap pembagian data.

3.2. Pembagian Fitur Berdasarkan Jenisnya

Sebelum melakukan preprocessing, semua fitur pada dataset dibagi menjadi beberapa kelompok berdasarkan karakteristiknya agar dapat diproses dengan cara yang paling tepat.

Fitur numerik kontinu (seperti nilai ujian, usia, dan indikator ekonomi) dikelompokkan ke dalam `num_features`. Fitur ordinal yang memiliki urutan logis (seperti urutan pendaftaran dan kualifikasi sebelumnya) dikelompokkan secara terpisah. Fitur biner (hanya memiliki dua nilai unik) diidentifikasi secara otomatis, sedangkan fitur kategorikal nominal (seperti status perkawinan, mode aplikasi, jurusan, dan pekerjaan orang tua) dikelompokkan ke dalam `cat_features` untuk kemudian di-encoding.

```
col = list(train.columns)
col.remove("Target")

num_features = ["Previous_qualification_(grade)",
                "Admission_grade",
                "Age_at_enrollment",
                "Curricular_units_1st_sem_(credited)",
                "Curricular_units_1st_sem_(enrolled)",
                "Curricular_units_1st_sem_(evaluations)",
                "Curricular_units_1st_sem_(approved)",
                "Curricular_units_1st_sem_(grade)",
                "Curricular_units_1st_sem_(without_evaluations)",
                "Curricular_units_2nd_sem_(credited)",
                "Curricular_units_2nd_sem_(enrolled)",
                "Curricular_units_2nd_sem_(evaluations)",
                "Curricular_units_2nd_sem_(approved)",
                "Curricular_units_2nd_sem_(grade)",
                "Curricular_units_2nd_sem_(without_evaluations)",
                "Unemployment_rate",
                "Inflation_rate",
                "GDP"]

ord_features = ["Application_order",
                "Previous_qualification"]

bin_features = [feat for feat in col if train[feat].nunique() == 2 and feat
↪  not in ord_features]

cat_features = ["Marital_status",
                "Application_mode",
                "Course",
                "Nacionality",
                "Mothers_qualification",
                "Fathers_qualification",
                "Mothers_occupation",
                "Fathers_occupation"]
```

3.3. *Z-score clipping*

Untuk mengurangi dampak outlier yang ekstrem pada fitur numerik tanpa menghapus data, diterapkan teknik *Z-score clipping*. Pada tahap `fit`, rata-rata (*mean*) dan standar deviasi (*std*) dari setiap kolom numerik dihitung dan disimpan. Pada tahap `transform`, nilai Z-score dihitung untuk setiap observasi, kemudian nilai Z-score tersebut dibatasi (clipped) pada rentang $[-\texttt{z_threshold}, \texttt{z_threshold}]$.

Setelah itu, nilai kembali dikonversi ke skala asli dengan mengalikan kembali standar deviasi dan menambah rata-rata. Pendekatan ini mempertahankan distribusi data sambil membatasi pengaruh *outlier* yang terlalu jauh dari pusat distribusi.

```
class Preprocessing(BaseEstimator, TransformerMixin):
    ...

    def fit(self, X, y=None):
        ...

        # mean & std untuk Z-score clipping
        self.zscore_params = {}
        for col in self.numeric_cols:
            mean = X[col].mean()
            std = X[col].std()
            self.zscore_params[col] = (mean, std)

        ...
        return self

    def transform(self, X):
        ...

        X_num = X[self.numeric_cols].astype(float)

        for col in self.numeric_cols:
            mean, std = self.zscore_params[col]
            z = (X_num[col] - mean) / std
            z = z.clip(-self.z_threshold, self.z_threshold)
            X_num[col] = z * std + mean

        ...
        return pd.concat([X_num, X_bin, X_cat], axis=1)
```

3.4. Standard Scaler

StandardScaler digunakan untuk menstandarisasi fitur numerik sehingga memiliki rata-rata nol dan standar deviasi satu (jika `with_mean=True` dan `with_std=True`).

Pada metode `fit`, dihitung rata-rata dan standar deviasi per kolom; pada `transform`, nilai dikurangi rata-rata dan (opsional) dibagi standar deviasi. Kelas ini juga menyediakan `inverse_transform` untuk mengembalikan data ke skala aslinya, serta menangani kasus vektor satu dimensi dan pembagian dengan standar deviasi nol (diganti dengan 1.0 agar tetap stabil).

```python
import numpy as np
from ..base import BaseEstimator, TransformerMixin


class StandardScaler(BaseEstimator, TransformerMixin):

    def __init__(self, with_mean=True, with_std=True):
        self.with_mean = with_mean
        self.with_std = with_std
        self.mean_ = None
        self.scale_ = None
        self.n_samples_seen_ = None
        self.fitted = False

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)

        if X.ndim == 1:
            X = X.reshape(-1, 1)

        self.n_samples_seen_ = X.shape[0]

        if self.with_mean:
            self.mean_ = np.mean(X, axis=0)
        else:
            self.mean_ = None

        if self.with_std:
            self.scale_ = np.std(X, axis=0)
            self.scale_[self.scale_ == 0] = 1.0
        else:
            self.scale_ = None

        self.fitted = True
        return self

    def transform(self, X):
        if not self.fitted:
            raise ValueError("StandardScaler must be fitted before
            ↪  transform()")

        X = np.asarray(X, dtype=float)

        if X.ndim == 1:
            X = X.reshape(-1, 1)

        X_transformed = X.copy()

        if self.with_mean:
            X_transformed = X_transformed - self.mean_

        if self.with_std:
            X_transformed = X_transformed / self.scale_

        return X_transformed

    def inverse_transform(self, X):
        if not self.fitted:
```

```
            raise ValueError("StandardScaler must be fitted before
            ↪  inverse_transform()")

        X = np.asarray(X, dtype=float)

        if X.ndim == 1:
            X = X.reshape(-1, 1)

        X_inv = X.copy()

        if self.with_std:
            X_inv = X_inv * self.scale_

        if self.with_mean:
            X_inv = X_inv + self.mean_

        return X_inv
```

3.5. One Hot Encoding

One-Hot Encoding digunakan untuk mengubah fitur kategorikal (nominal) menjadi representasi numerik berbasis biner. Setiap kategori pada suatu fitur dikonversi menjadi kolom baru, di mana nilai 1 menunjukkan keberadaan kategori tersebut dan nilai 0 menunjukkan ketiadaannya. Proses ini memastikan bahwa model tidak salah menginterpretasikan nilai kategori sebagai nilai ordinal. *Encoder* menyimpan daftar kategori untuk setiap kolom sehingga proses transformasi tetap konsisten pada data latih dan data uji.

```
import numpy as np
import pandas as pd
from .base import BaseEncoder


class OneHotEncoder(BaseEncoder):

    def __init__(self, columns=None, handle_unknown="ignore", dtype=float):
        self.columns = columns
        self.handle_unknown = handle_unknown
        self.dtype = dtype
        self.categories_ = {}
        self.feature_names_ = []
        self.fitted = False

    def fit(self, X):
        X = self._to_dataframe(X)

        if self.columns is None:
            self.columns = X.select_dtypes(include=["object",
            ↪  "category"]).columns.tolist()

        for col in self.columns:
            self.categories_[col] = np.unique(X[col].astype(str)).tolist()

        self.feature_names_ = []
        for col in self.columns:
```

```
            for cat in self.categories_[col]:
                self.feature_names_.append(f"{col}__{cat}")

        self.fitted = True
        return self

    def transform(self, X):
        if not self.fitted:
            raise ValueError("OneHotEncoder must be fitted before
            ↪  transform()")

        X = self._to_dataframe(X)
        encoded_arrays = []

        for col in self.columns:
            values = X[col].astype(str).values
            categories = self.categories_[col]
            mat = np.zeros((len(values), len(categories)),
            ↪  dtype=self.dtype)

            for i, val in enumerate(values):
                if val in categories:
                    j = categories.index(val)
                    mat[i, j] = 1.0
                else:
                    if self.handle_unknown == "error":
                        raise ValueError(f"Unknown category '{val}' in
                        ↪  column '{col}'")

            encoded_arrays.append(mat)

        if encoded_arrays:
            return np.hstack(encoded_arrays).astype(self.dtype)
        else:
            return np.zeros((len(X), 0), dtype=self.dtype)

    def fit_transform(self, X):
        return self.fit(X).transform(X)

    def inverse_transform(self, encoded):
        if not self.fitted:
            raise ValueError("OneHotEncoder must be fitted before
            ↪  inverse_transform()")

        encoded = np.asarray(encoded)
        idx = 0
        outputs = {}

        for col in self.columns:
            num_cats = len(self.categories_[col])
            slice_col = encoded[:, idx: idx + num_cats]
            idx += num_cats

            inv = []
            for row in slice_col:
                if np.sum(row) == 0:
                    inv.append(None)
                else:
```

```
                    j = np.argmax(row)
                    inv.append(self.categories_[col][j])

            outputs[col] = inv

        return pd.DataFrame(outputs)

    def _to_dataframe(self, X):
        if not isinstance(X, pd.DataFrame):
            return pd.DataFrame(X)
        return X
```

3.6. Label Encoding

Label Encoding digunakan untuk mengkonversi label kategorikal pada variabel target menjadi representasi integer. Setiap kelas diberikan indeks numerik sesuai urutan uniknya. Teknik ini sangat sesuai untuk label pada masalah klasifikasi serta fitur kategorikal yang digunakan pada model pohon (*tree-based*) karena model jenis ini tidak sensitif terhadap makna numerik yang diciptakan oleh *encoding*. Encoder juga menyimpan pemetaan dua arah (*class-to-int* dan *int-to-class*) untuk memudahkan proses interpretasi hasil prediksi.

```
import numpy as np
from .base import BaseEncoder


class LabelEncoder(BaseEncoder):

    def __init__(self):
        self.class_to_int = {}
        self.int_to_class = {}
        self.fitted = False

    def fit(self, y):
        classes = np.unique(y)
        self.class_to_int = {c: i for i, c in enumerate(classes)}
        self.int_to_class = {i: c for c, i in self.class_to_int.items()}
        self.fitted = True
        return self

    def transform(self, y):
        if not self.fitted:
            raise ValueError("LabelEncoder must be fitted before
            ↪  transform()")

        result = []
        for c in y:
            if c not in self.class_to_int:
                raise ValueError(f"Unknown label: {c}. Known labels:
                ↪  {list(self.class_to_int.keys())}")
            result.append(self.class_to_int[c])
        return np.array(result, dtype=int)

    def fit_transform(self, y):
        return self.fit(y).transform(y)
```

```
    def inverse_transform(self, y_int):
        if not self.fitted:
            raise ValueError("LabelEncoder must be fitted before
            ↪  inverse_transform()")
        return np.array([self.int_to_class[i] for i in y_int])
```

3.7. Preprocessing Pipeline

Preprocessing pipeline dirancang sebagai kelas `Preprocessing` yang mengintegrasikan seluruh langkah transformasi data dalam satu estimator yang kompatibel dengan scikit-learn.

Pada contoh implementasi, beberapa fitur kualifikasi orang tua di-drop karena redundansi atau korelasi rendah dengan target, sedangkan fitur ordinal seperti `Application_order` diperlakukan bersama fitur numerik untuk clipping dan scaling. Pipeline ini di-`fit` pada data latih, kemudian langsung menghasilkan data yang siap digunakan untuk pelatihan model.

```
label_encoder = LabelEncoder()
label_encoder.fit(train[settings.TARGET])

class Preprocessing(BaseEstimator, TransformerMixin):
    def __init__(self, initial_cols=None, numeric_cols=None,
    ↪  binary_cols=None, categorical_cols=None, dropped_cols=None,
    ↪  z_threshold=1.9):
        self.categorical_cols = categorical_cols or []
        self.binary_cols = binary_cols or []
        self.numeric_cols = numeric_cols or []
        self.dropped_cols = dropped_cols or []
        self.initial_cols = initial_cols or []

        self.ohe = OneHotEncoder(columns=self.categorical_cols,
        ↪  dtype=float)
        self.scaler = StandardScaler()
        self.fitted = False

        self.z_threshold = z_threshold
        self.zscore_params = {}

    def fit(self, X, y=None):
        if not isinstance(X, pd.DataFrame):
            raise ValueError("X must be a pandas DataFrame")

        if self.initial_cols:
            X = X[self.initial_cols]

        if self.dropped_cols:
            X = X.drop(columns=self.dropped_cols)

        # mean & std untuk Z-score clipping
        self.zscore_params = {}
        for col in self.numeric_cols:
            mean = X[col].mean()
            std = X[col].std()
            self.zscore_params[col] = (mean, std)
```

```python
        self.ohe.fit(X)
        self.scaler.fit(X[self.numeric_cols])

        self.fitted = True
        return self

    def transform(self, X):
        if not self.fitted:
            raise ValueError("Preprocessing must be fitted before
            ↪  transform()")

        if self.initial_cols:
            X = X[self.initial_cols]

        if self.dropped_cols:
            X = X.drop(columns=self.dropped_cols)

        X_num = X[self.numeric_cols].astype(float)

        # Z-score clipping
        for col in self.numeric_cols:
            mean, std = self.zscore_params[col]
            z = (X_num[col] - mean) / std
            z = z.clip(-self.z_threshold, self.z_threshold)
            X_num[col] = z * std + mean

        # categorical
        X_cat_array = self.ohe.transform(X)
        X_cat = pd.DataFrame(
            X_cat_array,
            columns=self.ohe.feature_names_,
            index=X.index
        )

        # binary features (no encoding needed)
        X_bin = X[self.binary_cols].astype(float)

        # scaling
        X_num_scaled = self.scaler.transform(X_num)
        X_num = pd.DataFrame(X_num_scaled, columns=self.numeric_cols,
        ↪  index=X.index)

        return pd.concat([X_num, X_bin, X_cat], axis=1)

initial_features = []
dropped_features = ["Mothers_qualification", "Fathers_qualification"]
cat_features = [col for col in cat_features if col not in dropped_features]

preprocessor = Preprocessing(
    initial_cols=initial_features,
    numeric_cols=num_features + ord_features,
    binary_cols=bin_features,
    categorical_cols=cat_features,
    dropped_cols=dropped_features,
    z_threshold=2.0
)
```

```
target = train[settings.TARGET]
train  = train.drop(columns=[settings.TARGET])
train  = preprocessor.fit_transform(train).copy(deep = True)
```

3.8. Feature Selection

Untuk mendukung berbagai metode seleksi fitur di tahap selanjutnya (seperti seleksi berbasis model, recursive feature elimination, atau seleksi berdasarkan importance), dibuat kelas abstrak `BaseFeatureSelector` yang mewarisi `BaseEstimator` dan `TransformerMixin` dari scikit-learn.

Kelas ini mendefinisikan antarmuka standar dengan metode `fit` (yang menerima data `X` dan target `y`) serta `transform` untuk menghasilkan hanya fitur-fitur yang terpilih. Pendekatan ini memungkinkan integrator pipeline scikit-learn yang konsisten dan memudahkan eksperimen dengan berbagai strategi seleksi fitur tanpa mengubah alur utama pipeline.

```
from abc import ABC, abstractmethod
from ..base import BaseEstimator, TransformerMixin


class BaseFeatureSelector(BaseEstimator, TransformerMixin, ABC):

    @abstractmethod
    def fit(self, X, y):
        pass

    @abstractmethod
    def transform(self, X):
        pass
```

Forward Feature Selection

Forward Feature Selection merupakan metode seleksi fitur yang dimulai dari himpunan kosong dan secara iteratif menambahkan satu fitur yang memberikan peningkatan performa model terbesar. Pada setiap iterasi, skor model dihitung menggunakan cross-validation dengan subset fitur yang telah dipilih ditambah satu kandidat fitur baru. Fitur dengan skor tertinggi dipilih hingga mencapai jumlah fitur target. Metode ini efektif untuk mengidentifikasi fitur yang paling informatif, meskipun mahal secara komputasi untuk dataset dengan fitur besar.

```
import numpy as np
from typing import Callable, Union
from concurrent.futures import ThreadPoolExecutor
from .base import BaseFeatureSelector
from ..metrics.base import Metric
from ..model_selection.base import BaseCrossValidator


class ForwardFeatureSelection(BaseFeatureSelector):
```

```python
    def __init__(
        self,
        estimator,
        n_features_to_select: int,
        scoring: Union[Metric, Callable],
        cv: BaseCrossValidator,
        n_jobs: int = 1,
        verbose: bool = True,
        feature_names: list = None
    ):
        self.estimator = estimator
        self.n_features_to_select = n_features_to_select
        self.scoring = scoring
        self.cv = cv
        self.n_jobs = n_jobs
        self.verbose = verbose
        self.feature_names = feature_names
        self.selected_features_ = None

    def _get_feature_name(self, idx):
        if self.feature_names is not None:
            return self.feature_names[idx]
        return f"Feature_{idx}"

    def _evaluate_features(self, X, y):
        scores = []
        for train_idx, val_idx in self.cv.split(X, y):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.estimator.fit(X_train, y_train)
            y_pred = self.estimator.predict(X_val)
            score = self.scoring(y_val, y_pred)
            scores.append(score)

        return np.mean(scores)

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        n_features = X.shape[1]

        if self.verbose:
            print("=" * 60)
            print("Forward Feature Selection")
            print(f"Total features: {n_features}")
            print(f"Target features: {self.n_features_to_select}")
            print("=" * 60)

        selected = []
        remaining = list(range(n_features))

        while len(selected) < self.n_features_to_select:
            iteration = len(selected) + 1
            best_score = -np.inf
            best_feature = None

            if self.verbose:
```

```
                print(f"\nIteration
                ↪  {iteration}/{self.n_features_to_select}")
                print(f"Testing {len(remaining)} features...")

            if self.n_jobs == 1:
                # Sequential execution
                for feature in remaining:
                    current_features = selected + [feature]
                    X_subset = X[:, current_features]
                    score = self._evaluate_features(X_subset, y)

                    if score > best_score:
                        best_score = score
                        best_feature = feature
            else:
                # Parallel execution
                def evaluate_feature(feature):
                    current_features = selected + [feature]
                    X_subset = X[:, current_features]
                    score = self._evaluate_features(X_subset, y)
                    return feature, score

                with ThreadPoolExecutor(max_workers=self.n_jobs) as
                ↪  executor:
                    results = list(executor.map(evaluate_feature,
                    ↪  remaining))

                for feature, score in results:
                    if score > best_score:
                        best_score = score
                        best_feature = feature

            selected.append(best_feature)
            remaining.remove(best_feature)

            if self.verbose:
                feature_name = self._get_feature_name(best_feature)
                print(f"Selected: {feature_name} (score:
                ↪  {best_score:.4f})")
                selected_names = [self._get_feature_name(f) for f in
                ↪  selected]
                print(f"Current features: {selected_names}")

        self.selected_features_ = np.array(selected)

        if self.verbose:
            print("\n" + "=" * 60)
            print("Feature selection completed!")
            selected_names = [self._get_feature_name(f) for f in
            ↪  self.selected_features_]
            print(f"Final selected features: {selected_names}")
            print("=" * 60)

        return self

    def transform(self, X):
        if self.selected_features_ is None:
            raise ValueError("You must call fit before transform")
```

```
        # Handle pandas DataFrame
        try:
            import pandas as pd
            if isinstance(X, pd.DataFrame):
                return X.iloc[:, self.selected_features_].values
        except ImportError:
            pass

        # Handle numpy array or convert to numpy
        X = np.asarray(X)
        return X[:, self.selected_features_]
```

Backward Feature Elimination

Backward Feature Elimination bekerja secara kebalikan dari FFS. Metode ini dimulai dari seluruh fitur yang tersedia, kemudian secara iteratif menghapus fitur yang memiliki kontribusi paling kecil terhadap performa model. Pada setiap iterasi, skor model dihitung tanpa salah satu fitur, dan fitur dengan penurunan skor terkecil dieliminasi. BFE sangat efektif pada dataset dengan korelasi antar fitur, meskipun lebih berat secara komputasi dibandingkan FFS.

```
import numpy as np
from typing import Callable, Union
from concurrent.futures import ThreadPoolExecutor
from .base import BaseFeatureSelector
from ..metrics.base import Metric
from ..model_selection.base import BaseCrossValidator


class BackwardFeatureElimination(BaseFeatureSelector):

    def __init__(
        self,
        estimator,
        n_features_to_select: int,
        scoring: Union[Metric, Callable],
        cv: BaseCrossValidator,
        n_jobs: int = 1,
        verbose: bool = True,
        feature_names: list = None
    ):
        self.estimator = estimator
        self.n_features_to_select = n_features_to_select
        self.scoring = scoring
        self.cv = cv
        self.n_jobs = n_jobs
        self.verbose = verbose
        self.feature_names = feature_names
        self.selected_features_ = None

    def _get_feature_name(self, idx):
        if self.feature_names is not None:
            return self.feature_names[idx]
        return f"Feature_{idx}"
```

```python
    def _evaluate_features(self, X, y):
        scores = []
        for train_idx, val_idx in self.cv.split(X, y):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.estimator.fit(X_train, y_train)
            y_pred = self.estimator.predict(X_val)
            score = self.scoring(y_val, y_pred)
            scores.append(score)

        return np.mean(scores)

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        n_features = X.shape[1]

        if self.verbose:
            print("=" * 60)
            print("Backward Feature Elimination")
            print(f"Total features: {n_features}")
            print(f"Target features: {self.n_features_to_select}")
            print("=" * 60)

        selected = list(range(n_features))

        while len(selected) > self.n_features_to_select:
            iteration = n_features - len(selected) + 1
            best_score = -np.inf
            least_important_feature = None

            if self.verbose:
                print(f"\nIteration {iteration}/{n_features -
                ↪  self.n_features_to_select}")
                print(f"Testing removal of {len(selected)} features...")

            if self.n_jobs == 1:
                # Sequential execution
                for feature in selected:
                    current_features = [f for f in selected if f !=
                    ↪  feature]
                    X_subset = X[:, current_features]
                    score = self._evaluate_features(X_subset, y)

                    if score > best_score:
                        best_score = score
                        least_important_feature = feature
            else:
                # Parallel execution
                def evaluate_removal(feature):
                    current_features = [f for f in selected if f !=
                    ↪  feature]
                    X_subset = X[:, current_features]
                    score = self._evaluate_features(X_subset, y)
                    return feature, score
```

```
            with ThreadPoolExecutor(max_workers=self.n_jobs) as
            ↪  executor:
                results = list(executor.map(evaluate_removal,
                ↪  selected))

            for feature, score in results:
                if score > best_score:
                    best_score = score
                    least_important_feature = feature

        selected.remove(least_important_feature)

        if self.verbose:
            feature_name =
            ↪  self._get_feature_name(least_important_feature)
            print(f"Removed: {feature_name} (score without it:
            ↪  {best_score:.4f})")
            remaining_names = [self._get_feature_name(f) for f in
            ↪  selected]
            print(f"Remaining features ({len(selected)}):
            ↪  {remaining_names}")

    self.selected_features_ = np.array(selected)

    if self.verbose:
        print("\n" + "=" * 60)
        print("Feature elimination completed!")
        selected_names = [self._get_feature_name(f) for f in
        ↪  self.selected_features_]
        print(f"Final selected features: {selected_names}")
        print("=" * 60)

    return self

def transform(self, X):
    if self.selected_features_ is None:
        raise ValueError("You must call fit before transform")

    # Handle pandas DataFrame
    try:
        import pandas as pd
        if isinstance(X, pd.DataFrame):
            return X.iloc[:, self.selected_features_].values
    except ImportError:
        pass

    # Handle numpy array or convert to numpy
    X = np.asarray(X)
    return X[:, self.selected_features_]
```

Bab IV
Desain Pemodelan

4.1. Cross Validation

Untuk mengevaluasi performa model secara lebih andal dan menghindari overfitting, dibuat kerangka kerja cross-validation yang fleksibel melalui kelas abstrak `BaseCrossValidator`.

Kelas ini mendefinisikan antarmuka utama berupa metode `split` yang harus diimplementasikan oleh subclass-nya. Pendekatan ini memungkinkan penggunaan berbagai strategi pembagian data (hold-out, k-fold, dll.) dengan cara yang konsisten dan mudah diganti sesuai kebutuhan eksperimen.

```
from abc import ABC, abstractmethod

class BaseCrossValidator(ABC):
    @abstractmethod
    def split(self, X, y=None):
        raise NotImplementedError("Subclasses should implement this
        ↪  method.")
```

Hold-out Validation

Implementasi pertama adalah `StratifiedHoldout`, yaitu metode hold-out yang menjaga proporsi kelas yang sama antara data latih dan data uji (stratified split). Hal ini sangat penting pada dataset yang tidak seimbang agar evaluasi performa model lebih representatif.

Metode `split` menghasilkan satu pasang indeks train/test dengan cara memisahkan sampel per kelas secara proporsional, kemudian mengacak ulang indeks dalam masing-masing set.

```
class StratifiedHoldout(BaseCrossValidator):
    """Stratified Hold-out cross-validator."""

    def __init__(self, test_size=0.2, shuffle=True, random_state=None):
        if not 0 < test_size < 1:
            raise ValueError(f"test_size must be between 0 and 1, got
            ↪  {test_size}")
        self.test_size = test_size
        self.shuffle = shuffle
        self.random_state = random_state

    def split(self, X, y):
        if y is None:
            raise ValueError("y is required for StratifiedHoldout")

        y = np.asarray(y)
        n_samples = len(X)
```

```
        unique_classes, y_inv = np.unique(y, return_inverse=True)
        n_classes = len(unique_classes)
        class_counts = np.bincount(y_inv)

        train_indices = []
        test_indices = []

        if self.shuffle:
            rng = np.random.RandomState(self.random_state)

        for class_idx in range(n_classes):
            class_mask = (y_inv == class_idx)
            class_indices = np.where(class_mask)[0]
            n_class_samples = len(class_indices)

            if self.shuffle:
                rng.shuffle(class_indices)

            n_test = int(n_class_samples * self.test_size)
            n_train = n_class_samples - n_test

            train_indices.extend(class_indices[:n_train])
            test_indices.extend(class_indices[n_train:])

        train_indices = np.array(train_indices)
        test_indices = np.array(test_indices)

        if self.shuffle:
            rng.shuffle(train_indices)
            rng.shuffle(test_indices)

        yield train_indices, test_indices

    def train_test_split(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)

        train_indices, test_indices = next(self.split(X, y))

        X_train = X[train_indices]
        X_test = X[test_indices]
        y_train = y[train_indices]
        y_test = y[test_indices]

        return X_train, X_test, y_train, y_test
```

K-fold Validation

Untuk memastikan bahwa proses evaluasi model tidak bias terhadap distribusi kelas yang tidak seimbang, digunakan Stratified K-Fold Cross-Validation. Metode ini membagi dataset menjadi K *fold*, tetapi tetap menjaga agar proporsi masing-masing kelas pada setiap *fold* mendekati distribusi keseluruhan. Dengan demikian, model diuji pada data yang representatif dan performa evaluasi menjadi lebih stabil. Setiap *fold* digunakan sebagai data uji satu kali, dan sebagai data latih sebanyak K−1 kali.

```python
class StratifiedKFold(BaseCrossValidator):
    """Stratified K-Fold cross-validator."""

    def __init__(self, n_splits=5, shuffle=False, random_state=None):
        if n_splits < 2:
            raise ValueError(f"n_splits must be at least 2, got
            ↪  {n_splits}")
        self.n_splits = n_splits
        self.shuffle = shuffle
        self.random_state = random_state

    def split(self, X, y):
        if y is None:
            raise ValueError("y is required for StratifiedKFold")

        y = np.asarray(y)
        n_samples = len(X)

        if self.n_splits > n_samples:
            raise ValueError(
                f"n_splits={self.n_splits} cannot be greater than "
                f"number of samples={n_samples}"
            )

        unique_classes, y_inv = np.unique(y, return_inverse=True)
        n_classes = len(unique_classes)
        class_counts = np.bincount(y_inv)

        if np.any(self.n_splits > class_counts):
            raise ValueError(
                f"n_splits={self.n_splits} cannot be greater than "
                f"the number of members in each class"
            )

        test_folds = np.zeros(n_samples, dtype=int)

        if self.shuffle:
            rng = np.random.RandomState(self.random_state)

        for class_idx in range(n_classes):
            class_mask = (y_inv == class_idx)
            class_indices = np.where(class_mask)[0]
            n_class_samples = len(class_indices)

            if self.shuffle:
                rng.shuffle(class_indices)

            fold_assignment = np.arange(n_class_samples) % self.n_splits
            test_folds[class_indices] = fold_assignment

        for fold_idx in range(self.n_splits):
            test_mask = (test_folds == fold_idx)
            test_indices = np.where(test_mask)[0]
            train_indices = np.where(~test_mask)[0]
            yield train_indices, test_indices

    def get_n_splits(self):
        return self.n_splits
```

4.2. Metric

Untuk menilai performa model pada dataset dengan distribusi kelas yang tidak seimbang, digunakan metrik F1-Score Macro sesuai pada Landasan Teori. Metrik ini menghitung nilai F1 secara independen untuk setiap kelas sebagai harmonisasi antara Precision dan Recall. Nilai F1 dari seluruh kelas kemudian dirata-ratakan tanpa bobot tambahan sehingga setiap kelas memiliki kontribusi yang sama, terlepas dari jumlah sampel masing-masing.

Metrik ini sangat cocok untuk mengukur akurasi model pada dataset imbalanced dan memberikan gambaran yang lebih adil mengenai performa model pada kelas minoritas.

```python
from .base import Metric
import numpy as np

class F1Score(Metric):
    def __init__(self, average="macro"):
        assert average in ["macro", "micro", "weighted"], "Invalid average"
        self.average = average

    def __call__(self, y_true, y_pred):
        y_true = np.asarray(y_true)
        y_pred = np.asarray(y_pred)

        labels = np.unique(np.concatenate([y_true, y_pred]))

        tp = np.array([np.sum((y_true == c) & (y_pred == c)) for c in
        ↪  labels])
        fp = np.array([np.sum((y_true != c) & (y_pred == c)) for c in
        ↪  labels])
        fn = np.array([np.sum((y_true == c) & (y_pred != c)) for c in
        ↪  labels])

        precision = np.where(tp + fp == 0, 0, tp / (tp + fp))
        recall    = np.where(tp + fn == 0, 0, tp / (tp + fn))

        f1 = np.where(
            precision + recall == 0,
            0,
            2 * precision * recall / (precision + recall)
        )

        if self.average == "macro":
            return float(np.mean(f1))

        elif self.average == "micro":
            tp_sum = tp.sum()
            fp_sum = fp.sum()
            fn_sum = fn.sum()
            prec = tp_sum / (tp_sum + fp_sum)
            rec  = tp_sum / (tp_sum + fn_sum)
            return float(2 * prec * rec / (prec + rec)) if (prec + rec) > 0
            ↪  else 0
        elif self.average == "weighted":
            weights = np.array([np.sum(y_true == c) for c in labels])
            return float(np.average(f1, weights=weights))
```

4.3. Implementasi Algoritma Model

Kelas abstrak `BaseClassifier` mewarisi `BaseEstimator` dan mendefinisikan antarmuka utama untuk model klasifikasi, yaitu metode `fit` untuk pelatihan dan `predict` untuk prediksi. Setiap algoritma klasifikasi yang diimplementasikan nantinya akan mewarisi `BaseClassifier` sehingga dapat langsung diintegrasikan ke dalam pipeline preprocessing dan evaluasi yang telah dibangun sebelumnya.

```
from abc import ABC, abstractmethod

class BaseEstimator:
    def get_params(self, deep=True):
        out = dict()
        for key in self._get_param_names():
            value = getattr(self, key, None)
            if deep and hasattr(value, 'get_params'):
                deep_items = value.get_params().items()
                out.update((key + '__' + k, val) for k, val in deep_items)
            out[key] = value
        return out

    def set_params(self, **params):
        if not params:
            return self
        valid_params = self.get_params(deep=True)

        for key, value in params.items():
            if key not in valid_params:
                raise ValueError(f'Invalid parameter {key} for estimator
                ↪  {self.__class__.__name__}')
            setattr(self, key, value)

        return self

    def _get_param_names(self):
        import inspect
        init = getattr(self.__init__, 'deprecated_original', self.__init__)
        init_signature = inspect.signature(init)
        parameters = [p for p in init_signature.parameters.values()
                      if p.name != 'self' and p.kind != p.VAR_KEYWORD]
        return sorted([p.name for p in parameters])


class TransformerMixin:
    def fit_transform(self, X, y=None, **fit_params):
        return self.fit(X, y, **fit_params).transform(X)


class BaseClassifier(BaseEstimator, ABC):
    @abstractmethod
    def fit(self, X, y):
        pass

    @abstractmethod
    def predict(self, X):
        pass
```

Decision Tree Classifier

Implementasi Decision Tree ini mendukung class weighting dan perhitungan entropi berbasis bobot. Pada fungsi fit(), model terlebih dahulu menyimpan kelas unik (self.classes_) lalu menghitung bobot setiap sampel jika class_weight diberikan. Jika opsi 'balanced' digunakan, bobot dihitung berbanding terbalik dengan frekuensi kelas sehingga kelas minoritas diberi bobot lebih tinggi. Bobot ini kemudian diteruskan ke metode fit() milik kelas induk agar seluruh proses pembentukan pohon menggunakan bobot tersebut.

Nilai prediksi pada node daun dihitung oleh _calculate_leaf_value(), yang memilih kelas dengan total weighted vote terbesar, bukan sekadar frekuensi terbanyak. Hal ini membuat prediksi lebih adil ketika dataset tidak seimbang.

Proses pemilihan split menggunakan weighted information gain melalui fungsi _information_gain(). Pertama, entropi induk dihitung menggunakan _entropy(), kemudian data dibagi berdasarkan threshold. Untuk sisi kiri dan kanan, entropi dihitung kembali menggunakan bobot sampel, dan gain diperoleh dari selisih antara entropi induk dan gabungan entropi anak yang sudah ditimbang menggunakan proporsi bobot pada masing-masing sisi.

Fungsi _entropy() menghitung entropi berdasarkan bobot dengan rumus: probabilitas setiap kelas dihitung dari jumlah bobot pada kelas tersebut dibagi total bobot pada node. Dengan cara ini, seluruh proses pemilihan threshold dan perhitungan impurity menjadi weight-sensitive

Secara keseluruhan, implementasi ini menghasilkan Decision Tree yang dapat menangani dataset imbalanced karena setiap langkah pemilihan split, perhitungan impurity, dan penentuan kelas akhir mempertimbangkan bobot sampel.

```
def __init__(self, max_depth=None, min_samples_split=2,
             min_samples_leaf=1, random_state=None, class_weight=None):
    super().__init__(max_depth, min_samples_split, min_samples_leaf,
    ↪  random_state)
    self.class_weight = class_weight
```

Fungsi ini digunakan untuk mengatur parameter awal model seperti max_depth, jumlah minimum sampel untuk split, dan pengaturan class_weight untuk menangani data tidak seimbang. Nilai parameter diteruskan ke induk sehingga Decision Tree mengikuti konfigurasi tersebut selama pelatihan.

```
def fit(self, X, y):
    self.classes_ = np.unique(y)
    sample_weight = self._compute_sample_weight(y) if self.class_weight is
    ↪  not None else None
    super().fit(X, y, sample_weight=sample_weight)
    return self
```

Fungsi fit() memulai proses pelatihan dengan menyimpan daftar kelas unik lalu menghitung bobot sampel jika pengguna mengaktifkan class weighting. Data bersama bobotnya kemudian diberikan ke metode fit milik kelas induk sehingga seluruh proses pembentukan pohon mempertimbangkan bobot tersebut.

```python
def _compute_sample_weight(self, y):
    if self.class_weight == "balanced":
        class_counts = np.bincount(y)
        n_samples = len(y)
        n_classes = len(self.classes_)
        weights = n_samples / (n_classes * class_counts)
        return weights[y]
    if isinstance(self.class_weight, dict):
        return np.array([self.class_weight.get(cls, 1.0) for cls in y])
    return np.ones(len(y))
```

Fungsi ini menghasilkan bobot untuk setiap sampel, baik secara otomatis berdasarkan proporsi kelas (balanced) ataupun berdasarkan nilai manual dari dictionary. Jika class_weight tidak digunakan, semua sampel diberi bobot 1.

```python
def _calculate_leaf_value(self, y, sample_weight):
    weighted_votes = {}
    for cls, w in zip(y, sample_weight):
        weighted_votes[cls] = weighted_votes.get(cls, 0) + w
    return max(weighted_votes, key=weighted_votes.get)
```

Fungsi ini menentukan kelas pada node daun dengan memilih kelas berdasarkan jumlah bobot terbesar, bukan frekuensi terbanyak. Mekanisme ini membuat hasil prediksi tetap fair meskipun data tidak seimbang.

```python
def _information_gain(self, y, X_column, threshold, sample_weight):
    parent_entropy = self._entropy(y, sample_weight)
    left = X_column < threshold
    right = ~left
    if np.sum(left) == 0 or np.sum(right) == 0: return 0
    total_w = np.sum(sample_weight)
    w_left = np.sum(sample_weight[left])
    w_right = np.sum(sample_weight[right])
    e_left = self._entropy(y[left], sample_weight[left])
    e_right = self._entropy(y[right], sample_weight[right])
    child_entropy = (w_left / total_w) * e_left + (w_right / total_w) *
    ↪  e_right
    return parent_entropy - child_entropy
def _entropy(self, y, sample_weight):
    unique = np.unique(y)
    total_w = np.sum(sample_weight)
    entropy = 0.0
    for cls in unique:
        mask = (y == cls)
        w = np.sum(sample_weight[mask])
        if w > 0:
            p = w / total_w
            entropy -= p * np.log2(p)
    return entropy
```

Fungsi ini menghitung information gain untuk menentukan threshold terbaik pada split dengan membandingkan nilai entropi antara node induk dan anak, menggunakan bobot sampel sebagai dasar pembobotan impurity. Semakin besar

gain, semakin baik threshold tersebut sebagai pemisah data.

```
def _entropy(self, y, sample_weight):
    unique = np.unique(y)
    total_w = np.sum(sample_weight)
    entropy = 0.0
    for cls in unique:
        mask = (y == cls)
        w = np.sum(sample_weight[mask])
        if w > 0:
            p = w / total_w
            entropy -= p * np.log2(p)
    return entropy
```

Fungsi ini menghitung nilai impurity node berdasarkan distribusi bobot kelas menggunakan rumus entropi. Dengan cara ini, nilai impurity memperhitungkan ketidakseimbangan dataset, bukan sekadar jumlah frekuensi.

Logistic Regression

Implementasi Logistic Regression menggunakan fungsi sigmoid sebagai inti perhitungan probabilitas, yaitu logistic function yang memetakan nilai linear menjadi rentang [0,1]. Sebelum proses pelatihan dimulai, model menghitung bobot sampel apabila opsi class_weight diberikan, sehingga kelas dengan frekuensi rendah mendapatkan kontribusi lebih besar dalam proses pembaruan parameter.

Pembaruan bobot dilakukan melalui prosedur optimisasi berbasis gradien seperti Stochastic Gradient Ascent (SGA), Batch Gradient Ascent (BGA), atau Mini-batch Gradient Descent, tergantung solver yang dipilih. Pada setiap iterasi, model menghitung prediksi, error berbobot, serta gradien, kemudian memperbarui vektor parameter menggunakan laju pembelajaran. Proses ini berhenti ketika gradien sudah sangat kecil atau jumlah iterasi maksimum tercapai. Untuk kasus multiclass,

Logistic Regression menerapkan pendekatan One-vs-Rest, yaitu melatih satu model biner untuk setiap kelas. Pada tahap inferensi, model menghitung logit untuk seluruh kelas, mengaplikasikan fungsi sigmoid, dan memilih kelas dengan probabilitas tertinggi sebagai hasil prediksi.

```
def _logistic_function(self, z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
```

Fungsi ini menerapkan sigmoid untuk mengubah nilai linear menjadi probabilitas dalam rentang 0–1, dengan clipping untuk menghindari overflow numerik.

```
def _compute_sample_weight(self, y):
    if self.class_weight == "balanced":
        classes, counts = np.unique(y, return_counts=True)
```

```
        w = len(y) / (len(classes) * counts)
        return w[np.searchsorted(classes, y)]
    elif isinstance(self.class_weight, dict):
        return np.array([self.class_weight.get(cls, 1.0) for cls in y])
    return np.ones(len(y))
```

Fungsi ini menghitung bobot setiap label agar kelas minoritas mendapat pengaruh lebih besar pada saat perhitungan gradien.

```
def _stochastic_gradient_ascent(self, X, y, sample_weight=None):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)


    if sample_weight is None:
        sample_weight = np.ones(n_samples)


    for _ in range(self.max_iter):
        indices = self._rng.permutation(n_samples)
        for start in range(0, n_samples, self.batch_size):
            batch_idx = indices[start:start + self.batch_size]
            X_batch = X[batch_idx]
            y_batch = y[batch_idx]
            w_batch = sample_weight[batch_idx]


            p = self._logistic_function(X_batch @ w)
            error = (y_batch - p) * w_batch
            gradient = X_batch.T @ error / w_batch.sum()
            w += self.learning_rate * gradient
            self.history.append(w.copy())


        p_full = self._logistic_function(X @ w)
        error_full = (y - p_full) * sample_weight
        gradient_full = X.T @ error_full / sample_weight.sum()
        if np.linalg.norm(gradient_full) < self.tol:
            break


    return w
```

Fungsi ini melakukan proses optimasi dengan menghitung gradien berdasarkan error berbobot dan memperbarui parameter hingga konvergen atau mencapai batas iterasi.

```
def fit(self, X, y):
    X = np.hstack([np.ones((len(X), 1)), np.asarray(X)])
    sample_weight = self._compute_sample_weight(y)
    self._classes = np.unique(y)
    if len(self._classes) == 2:
        y_bin = (y == self._classes[-1]).astype(int)
        self._W = self._batch_gradient_ascent(X, y_bin, sample_weight)
```

```
    else:
        self._W = np.vstack([
            self._batch_gradient_ascent(X, (y == cls).astype(int),
            ↪  sample_weight)
            for cls in self._classes
        ])
    return self
```

Dalam logistic regression, selain stochastic gradient ascent kami juga membuat beberapa jenis optimasi lain, yakni batch gradient ascent yang merupakan versi lebih sederhana dari stochastic gradient ascent, MGD (mini-batch gradient descent) dan Newton-CGA. Untuk MGD, kami terinspirasi dari [5] dan untuk Newton-CGA kami terinspirasi dari [6]. Untuk penjelasan lebih lanjut mengenai kedua optimasi tersebut dapat dilihat pada bagian Bonus.

```
def predict(self, X):
    X = np.hstack([np.ones((len(X), 1)), np.asarray(X)])
    logits = X @ self._W.T if self._W.ndim == 2 else X @ self._W
    probs = self._logistic_function(logits)
    return self._classes[np.argmax(probs, axis=1)] if probs.ndim == 2 else
    ↪  (probs > 0.5)
```

Fungsi ini melakukan pelatihan logistic regression, baik model biner maupun multiclass menggunakan pendekatan One-vs-Rest

```
def fit(self, X, y):
    ...
        if self._classes.size > 2:
            W_all = []
            for cls in self._classes:
                y_binary = (y == cls).astype(int)
                w = self.solver_fn(X, y_binary, sample_weight)
                W_all.append(w)
            self._W = np.vstack(W_all)
    ...
```

Support Vector Classifier

Implementasi SVM dimulai dengan pemilihan fungsi kernel, seperti linear, RBF, polynomial, atau sigmoid, yang menentukan bagaimana data diproyeksikan ke ruang fitur berdimensi lebih tinggi. Model kemudian membentuk matriks kernel, yaitu matriks yang menyimpan nilai kesamaan antar seluruh pasangan sampel, sebagai bagian dari formulasi problem optimisasi dual SVM.

Untuk mengatasi data tidak seimbang, bobot sampel dihitung berdasarkan class_weight, dan nilai penalti C diperbarui untuk tiap sampel. Proses pelatihan menggunakan pendekatan optimisasi seperti CVXOPT (Quadratic Programming), SMO, atau Pegasos, tergantung parameter optimizer. *Solver* tersebut menghitung Lagrange multipliers (α), dari mana *support vectors* ditentukan yaitui sampel dengan nilai α signifikan.

Bias model dihitung menggunakan rata-rata kesalahan pada *support vectors*. Jika kernel linear digunakan, bobot www dapat dihitung secara eksplisit, sedangkan kernel non-linear menggunakan bentuk dual sepenuhnya.

Pada tahap prediksi, model menghitung nilai fungsi keputusan berdasarkan kombinasi $\alpha_i \cdot y_i \cdot K(x, x_i)$ dan bias, kemudian menentukan label berdasarkan tanda (*sign*) dari hasil tersebut.

```
def _get_kernel_fn(self):
    if self.kernel == "linear":
        return lambda a, b: np.dot(a, b)
    if self.kernel == "rbf":
        return lambda a, b: np.exp(-self.gamma * np.dot(a - b, a - b))
    if self.kernel == "poly":
        return lambda a, b: (self.gamma * np.dot(a, b) + self.coef0) **
        ↪  self.degree
    if self.kernel == "sigmoid":
        return lambda a, b: np.tanh(self.gamma * np.dot(a, b) + self.coef0)
    raise ValueError("Unknown kernel.")
```

Fungsi ini memilih kernel yang akan digunakan (linear, RBF, polynomial, atau sigmoid), yang kemudian menentukan bagaimana data direpresentasikan dalam ruang fitur.

```
def _compute_kernel_matrix(self, X, Kfn):
    n = len(X)
    K = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            K[i, j] = Kfn(X[i], X[j])
    return K
```

Fungsi ini menghitung matriks kernel yang menyimpan nilai kesamaan antar sampel dan menjadi inti optimisasi dual SVM.

```
def fit(self, X, y):
    X = np.asarray(X, float)
    y = np.where(y == np.unique(y)[0], -1, 1)


    Kfn = self._get_kernel_fn()
    K = self._compute_kernel_matrix(X, Kfn)
    n = len(X)


    P = cvxopt.matrix(np.outer(y, y) * K)
    q = cvxopt.matrix(-np.ones(n))
    G = cvxopt.matrix(np.vstack([-np.eye(n), np.eye(n)]))
    h = cvxopt.matrix(np.hstack([np.zeros(n), np.ones(n) * self.C]))
    A = cvxopt.matrix(y.reshape(1, -1), tc='d')
    b = cvxopt.matrix(0.0)
```

```
    cvxopt.solvers.options['show_progress'] = False
    sol = cvxopt.solvers.qp(P, q, G, h, A, b)
    lamb = np.ravel(sol['x'])


    sv = lamb > 1e-5
    self.support_vectors_ = X[sv]
    self.support_labels_ = y[sv]
    self.dual_coef_ = lamb[sv]


    self.intercept_ = np.mean([
        yi - np.sum(self.dual_coef_ * self.support_labels_ *
                    np.array([Kfn(xi, svx) for svx in
                    ↪  self.support_vectors_]))
        for xi, yi in zip(self.support_vectors_, self.support_labels_)
    ])


    if self.kernel == "linear":
        self.coef_ = np.sum(
            self.dual_coef_.reshape(-1, 1) *
            ↪  self.support_labels_.reshape(-1, 1) *
            ↪  self.support_vectors_,
            axis=0
        )
    else:
        self.coef_ = None


    return self
```

Dalam support vector machine, selain fungsi di atas yang menyelesaian permasalahan dengan CVXOPT (Quadratic Programming), dibuat juga beberapa jenis optimasi lain, yakni SMO dan Pegasos, tergantung parameter optimizer. Untuk SMO, kami terinspirasi dari [7] dan untuk Pegasos kami terinspirasi dari [8].

```
def predict(self, X):
    X = np.asarray(X)
    Kfn = self._get_kernel_fn()
    decision = np.array([
        np.sum(self.dual_coef_ * self.support_labels_ *
               np.array([Kfn(x, svx) for svx in self.support_vectors_])) +
               ↪  self.intercept_
        for x in X
    ])
    return np.where(decision < 0, -1, 1)
```

Fungsi prediksi menghitung fungsi keputusan berdasarkan support vectors dan kernel, lalu menentukan kelas berdasarkan tanda hasil tersebut.

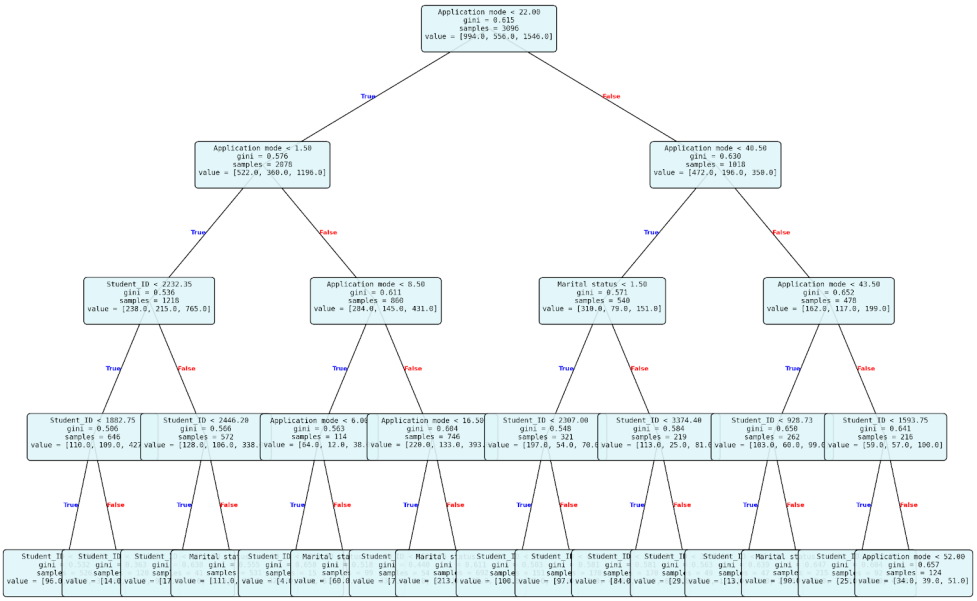
4.4. Implementasi Bonus

Pada tugas besar 2 AI ini, ada beberapa bonus yang dikerjakan dan akan dibahas:

Visualisasi Decision Tree

Pembuatan gambar ini dilakukan untuk melihat bagaimana model ini melakukan split atau pengklasifikasian sample di tiap kedalamannya. Untuk mengimplementasikan ini, kita membutuhkan tambahan atribut pada tiap node, yaitu impurity (menyimpan gini ratio entropy score) , n_samples (menyimpan nilai seberapa banyak data training yang sampai hingga node tersebut, dan juga self.value untuk mengetahui kecondongan ke tiap kelas. Inti dari cara kerja visualisasi ini ada pada fungsi _plot_node yang bekerja secara rekursif menuruni node-node dari decision tree dan membuat edge

```
def _plot_node(self, ax, node, x, y, dx, dy, depth, max_depth):
        ...

        # Hal-hal yang akan ditulis di visualisasi
        content = f"{self.criterion} = {node.impurity:.3f}\n"
        content += f"samples = {node.n_samples}\n"
        content += f"value = {val_str}\n"

        ...

        if (node.left is None and node.right is None) or depth >=
        ↪  max_depth:
            return

        ...

        # Gambar edge
        ax.plot([x, x_left], [y, y_next], 'k-', lw=1, zorder=-1)
        ax.plot([x, x_right], [y, y_next], 'k-', lw=1, zorder=-1)

        mid_y = (y + y_next) / 2
        mid_x_left = (x + x_left) / 2
        mid_x_right = (x + x_right) / 2

        ...

        # bagian rekursifnya

        self._plot_node(ax, node.left, x_left, y_next, dx/2, dy, depth + 1,
        ↪  max_depth)
        self._plot_node(ax, node.right, x_right, y_next, dx/2, dy, depth +
        ↪  1, max_depth)
```

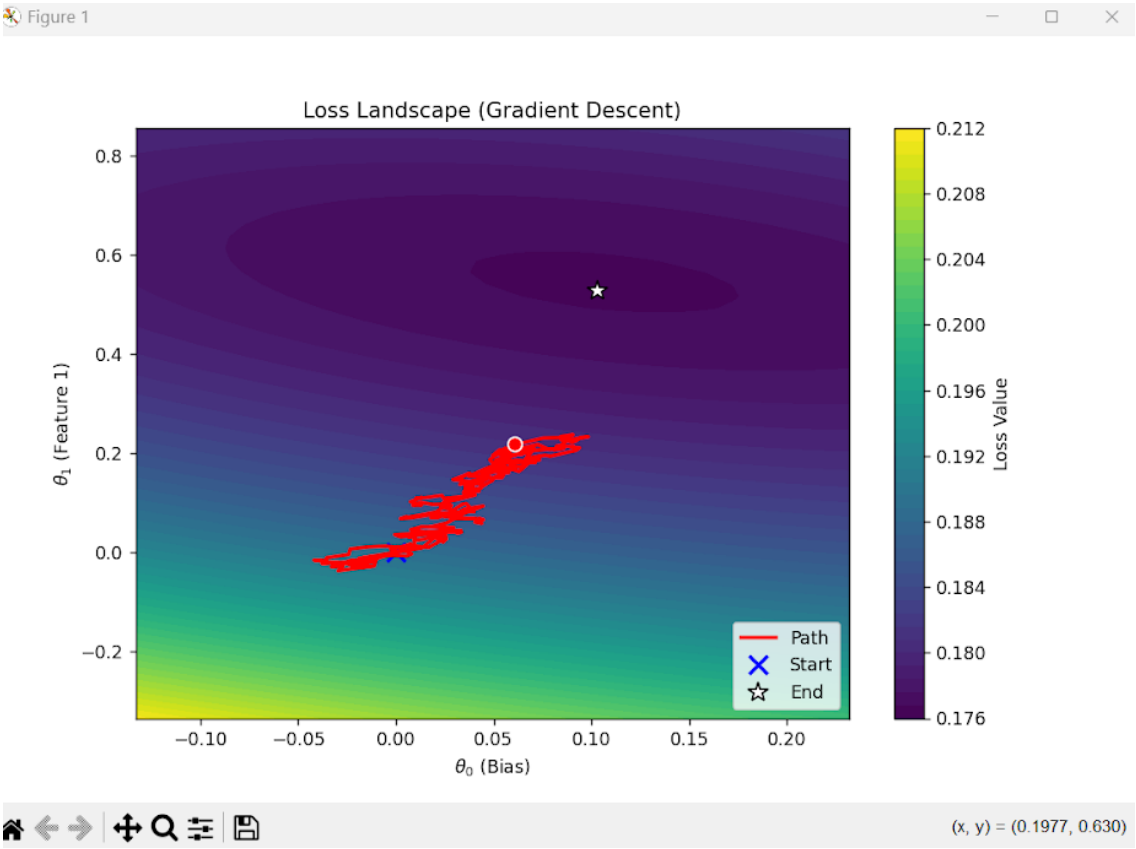


Gambar 7: Visualisasi Decision Tree

Video Pelatihan Logistic Regression

Pembuatan gambar ini dilakukan untuk melihat bagaimana model ini melakukan split atau pengklasifikasian sample di tiap kedalamannya. Untuk mengimplementasikan ini, kita membutuhkan tambahan atribut pada tiap node, yaitu impurity (menyimpan gini ratio entropy score) , n_samples (menyimpan nilai seberapa banyak data training yang sampai hingga node tersebut, dan juga self.value untuk mengetahui kecondongan ke tiap kelas. Inti dari cara kerja visualisasi ini ada pada fungsi _plot_node yang bekerja secara rekursif menuruni node-node dari decision tree dan membuat edge

```
def visualize_multiclass(model, X, y, save_path=None):
    ...
    # Penghitungan Z
    Z = np.zeros_like(W0)
    for i in range(W0.shape[0]):
        for j in range(W0.shape[1]):
            w_tmp = final_w.copy()
            w_tmp[0], w_tmp[1] = W0[i, j], W1[i, j]
            Z[i, j] = compute_metric(w_tmp, X_bias, y, mode=mode)
    # Peta Kontur
    fig, ax = plt.subplots(figsize=(9, 6))
    CS = ax.contourf(W0, W1, Z, 50, cmap=config['cmap'])
    …
    # untuk update frame
    def update(frame):
            line.set_data(w0_h[:frame+1], w1_h[:frame+1])
            point.set_data([w0_h[frame]], [w1_h[frame]])
            return line, poin
    ani = FuncAnimation(fig, update, frames=len(history), blit=True,
    ↪  interval=50)
```



Gambar 8: Visualisasi Pelatihan Logistic Regression

Optimisasi Logistic Regression

Dalam logistic regression, kami menyediakan empat pilihan optimizer. Batch gradient ascent adalah yang paling sederhana dan stabil: pada setiap iterasi seluruh data pelatihan digunakan untuk menghitung gradien secara eksak, lalu parameter langsung diperbarui satu kali; cocok untuk dataset kecil sampai menengah meskipun lambat pada data besar.

Stochastic gradient ascent melakukan kebalikannya, yaitu memperbarui bobot setelah melihat hanya satu sampel acak tiap langkah, sehingga sangat cepat dan bisa menangani dataset sangat besar walaupun trajektori konvergensinya lebih berisik.

Mini-batch gradient ascent (MGD) mengambil jalan tengah dengan menggunakan 32–256 sampel acak tiap iterasi, ini mengurangi noise dibanding stochastic sekaligus tetap jauh lebih efisien daripada batch penuh (terinspirasi dari [5]).

Terakhir, Newton-Conjugate Gradient Ascent (Newton-CGA) adalah metode orde dua yang memanfaatkan Hessian untuk menghitung arah pembaruan yang lebih akurat; konvergensinya jauh lebih cepat (sering hanya butuh puluhan iterasi) meskipun tiap iterasinya lebih mahal secara komputasi (terinspirasi dari [6]).

Optimisasi Support Vector Machine

Selain menyelesaikan permasalahan SVM hard-margin dan soft-margin dengan CVXOPT (quadratic programming eksak), kami juga mengimplementasikan dua algoritma optimasi populer yang jauh lebih cepat pada dataset besar.

Sequential Minimal Optimization (SMO) memecah masalah QP besar menjadi sub-masalah terkecil yang melibatkan hanya dua Lagrange multiplier sekaligus dengan heuristik pemilihan pasangan yang baik, SMO sangat efisien dan hingga kini menjadi standar industri untuk SVM klasik (terinspirasi dari [7]).

Pegasos (Primal Estimated sub-GrAdient SOlver for SVM) bekerja langsung di ruang primal dengan stochastic sub-gradient descent dan proyeksi sederhana; hanya butuh operasi vektor tiap iterasi sehingga sangat cepat dan mudah diskalakan ke jutaan sampel (terinspirasi dari [8]). Pilihan optimizer dapat diatur lewat parameter optimizer saat memanggil kelas SVM.

Bab V
Hasil dan Analisis

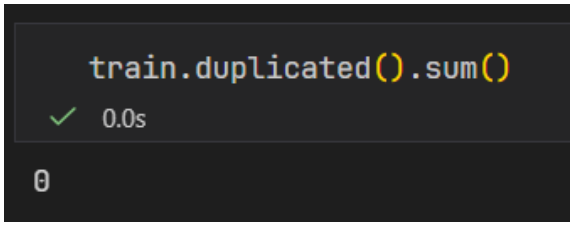
5.1. Exploratory Data Analysis

Analisis Data Eksplorasi (EDA) adalah proses pemeriksaan dan peringkasan dataset untuk mengungkap pola, tren, dan anomali. Dengan menggunakan visualisasi seperti histogram dan grafik lainnya, serta ukuran statistik seperti rata-rata dan korelasi.

EDA membantu memahami struktur data dan hubungan antar variabel. Proses ini mengidentifikasi outlier dan nilai yang hilang, memandu pembersihan data, serta memberikan wawasan untuk analisis atau pemodelan selanjutnya.

Baris Duplikat

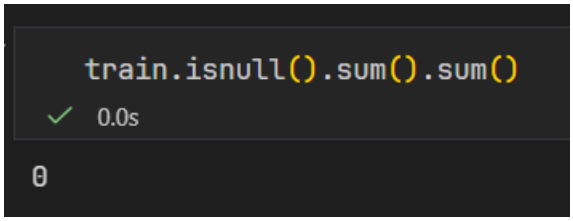
Tidak ditemukan adanya baris duplikat pada dataset, yang menunjukkan kualitas data yang baik dan mengurangi risiko bias dalam pelatihan model. Oleh karena itu, tidak diperlukan langkah penghapusan duplikat sehingga proses preprocessing dapat langsung fokus pada aspek lain seperti normalisasi dan encoding.



Gambar 9: Hasil Pengecekan Duplikasi Data

Missing Values

Tidak ada nilai *null* pada dataset, yang berarti dataset telah bersih dari permasalahan pemrosesan data awal. Insight ini memungkinkan tim untuk melewati tahap imputasi data hilang dan langsung melanjutkan ke analisis distribusi serta transformasi fitur sehingga mempercepat alur kerja keseluruhan.

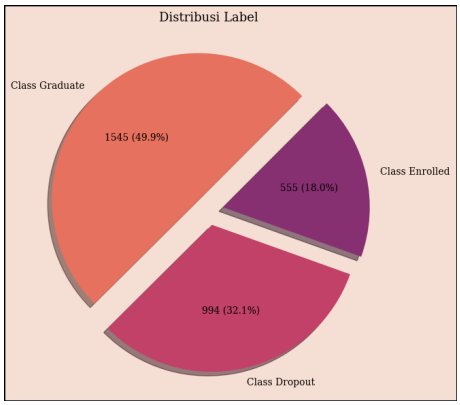


Gambar 10: Hasil Pengecekan Missing Values

Persebaran Label

Terlihat bahwa kelas ”Graduate” mendominasi daripada kelas lainnya, menyebabkan *imbalanced class distribution* yang dapat membuat model bias terhadap kelas mayoritas. Untuk mengatasi hal ini, diterapkan teknik *class weight* pada proses pelatihan model, di mana bobot kelas minoritas (seperti ”Dropout” dan ”Enrolled”) ditingkatkan secara proporsional terhadap frekuensinya dibandingkan kelas mayoritas.

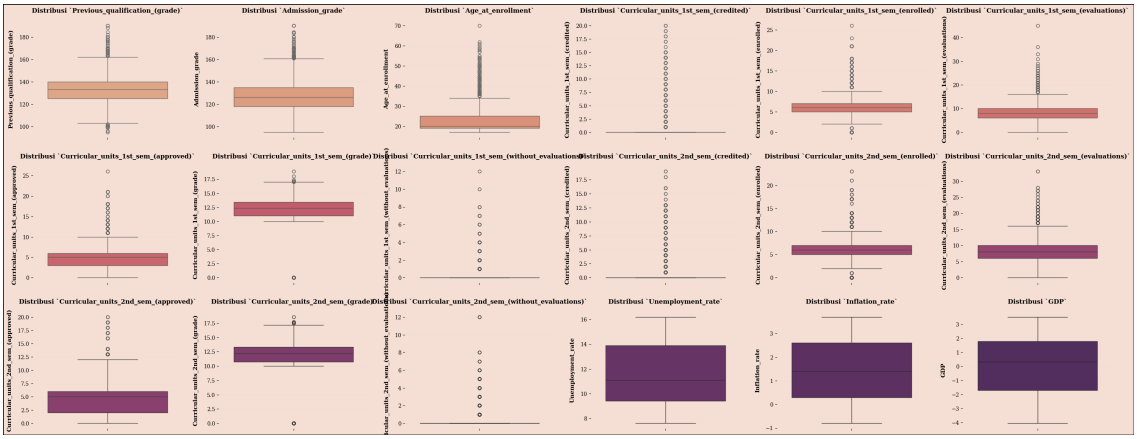
Pendekatan ini membuat fungsi loss memberikan penalti lebih besar saat salah mengklasifikasikan sampel dari kelas minoritas, sehingga meningkatkan kemampuan model dalam mendeteksi kelas minoritas dan menghindari prediksi yang tidak akurat pada kasus ambigu tanpa mengubah distribusi data asli.



Gambar 11: Distribusi Label pada Dataset

Analisis Outlier

Beberapa fitur memiliki *outlier* yang signifikan, yang dapat memengaruhi performa model dengan mendistorsi distribusi data. Langkah pengatasannya meliputi penerapan *Z-score clipping* untuk membatasi nilai ekstrem tanpa menghapus data sehingga menjaga integritas dataset sambil mengurangi dampak negatif outlier terhadap akurasi prediksi model.



Gambar 12: Visualisasi Boxplot pada Fitur Numerikal

Statistik Post-processing

Setelah dilakukan Pipeline Preprocessing, seluruh fitur numerikal terstandarisasi sesuai distribusinya dan juga menghasilkan 157 kolom setelah one-hot encoding, yang memerlukan *feature selection* untuk mengeliminasi fitur noise yang dapat menurunkan performa model.

Dilakukan *feature selection* dan diperoleh 49 fitur untuk Decision Tree Learning, 50 untuk Logistic Regression, dan 15 untuk Support Vector Machine; insight ini mengoptimalkan efisiensi komputasi dan meningkatkan generalisasi model dengan fokus pada fitur paling relevan.

5.2. Pelatihan dan Validasi Model

Untuk menguji performa model yang kami buat, kami bandingkan hasil *preprocessing* terbaik kami yang telah dilakukan bagi setiap model dan membandingkannya dengan model yang terdapat pada library Scikit-Learn. Tiap model akan melalui proses *preprocessing* yang sama hingga pada titik transformasi. Lalu tiap model akan dilakukan ForwardFeatureSelection untuk memilih fitur terbaik pada data. Fitur tersebut yang akan digunakan untuk proses latih dan prediksi.

Proses ini memastikan konsistensi evaluasi, di mana kami dapat mengidentifikasi kelemahan model manual dibandingkan implementasi standar. Jika performa rendah, langkah selanjutnya adalah mengoptimasi hyperparameter, seperti `max_depth` dan `min_sample_split` pada Decision Tree Learning yang pada akhirnya meningkatkan generalisasi model pada data baru.

Decision Tree Learning

```
{'accuracy': 0.7524271844660194,
 'f1_macro': 0.6810647241465032,
 'f1_micro': 0.7524271844660194,
 'f1_weighted': 0.7407265284892877}
```

Gambar 13: Hasil Hold-out validation pada DTL Manual

```
(array([[143,  17,  38],
        [ 21,  42,  48],
        [ 11,  18, 280]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 14: Confusion Matrix Hold-out validation pada DTL Manual

```
Aggregated results for model Decision Tree Manual:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.763285  0.701429  0.763285     0.758797
1  0.718901  0.646815  0.718901     0.715055
2  0.770598  0.706280  0.770598     0.764697
3  0.777060  0.729707  0.777060     0.777342
4  0.749191  0.651637  0.749191     0.727966
Mean metrics: {'accuracy': 0.755806985748046, 'f1_macro': 0.6871736891114696,
============================================================
```

Gambar 15: Hasil Hold-out validation pada DTL Manual

Pada DTL manual hold-out validation, didapat F1-score 0.6817. Pada K-fold validation, didapat 0.68717. Skor ini menunjukkan potensi *underfitting* pada data pelatihan. untuk mengatasinya, dilakukan *pruning* pohon dengan membatasi kedalaman maksimal atau minimum sampel per leaf, yang dapat meningkatkan stabilitas dan mengurangi varians.

```
{'accuracy': 0.7556634304207119,
 'f1_macro': 0.6883526166749353,
 'f1_micro': 0.7556634304207119,
 'f1_weighted': 0.7466400055516901}
```

Gambar 16: Hasil Hold-out validation pada DTL Scikit-learn

```
(array([[146,  19,  33],
        [ 23,  45,  43],
        [ 13,  20, 276]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 17: Confusion Matrix Hold-out validation pada DTL Scikit-learn

```
Aggregated results for model Decision Tree Sklearn:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.763285  0.701055  0.763285     0.758807
1  0.718901  0.646971  0.718901     0.715596
2  0.773829  0.710413  0.773829     0.768021
3  0.777060  0.729707  0.777060     0.777342
4  0.747573  0.650130  0.747573     0.727005
Mean metrics: {'accuracy': 0.7561295647066962, 'f1_macro': 0.687655208327855,
============================================================
```

Gambar 18: Hasil Hold-out validation pada DTL Scikit-learn

Pada DTL sklearn hold-out validation 0.6883, didapat k-fold validation f1-score 0.6876. Performa yang lebih baik ini disebabkan oleh optimasi internal library.

Logistic Regression

```
{'accuracy': 0.7815533980582524,
 'f1_macro': 0.6963129905623545,
 'f1_micro': 0.7815533980582524,
 'f1_weighted': 0.7627396294453287}
```

Gambar 19: Hasil Hold-out validation pada LogRes Manual

```
(array([[158,   9,  31],
        [ 29,  36,  46],
        [  8,  12, 289]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 20: Confusion Matrix Hold-out validation pada LogRes Manual

```
Aggregated results for model Logistic Regression:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.789050  0.700033  0.789050     0.769329
1  0.756058  0.664817  0.756058     0.736957
2  0.794830  0.708375  0.794830     0.775885
3  0.786753  0.709037  0.786753     0.771951
4  0.749191  0.659447  0.749191     0.727417
Mean metrics: {'accuracy': 0.7751764430046986, 'f1_macro': 0.6883420319570915,
============================================================
```

Gambar 21: Hasil Hold-out validation pada LogRes Manual

Implementasi manual Logistic Regression menggunakan `newton-cg` menghasilkan F1-score hold-out 0.6963 dan 10-fold CV 0.6883. Penurunan pada CV mengindikasikan sedikit underfitting akibat kurangnya regularisasi eksplisit. Penambahan penalti L2 (ridge) dengan `C=1.0` hingga `C=0.5` dapat menaikkan CV score.

```
{'accuracy': 0.7783171521035599,
 'f1_macro': 0.7055762196652801,
 'f1_micro': 0.77831715210356,
 'f1_weighted': 0.7649748008859937}
```

Gambar 22: Hasil Hold-out validation pada LogRes Scikit-learn

```
(array([[155,  11,  32],
        [ 25,  42,  44],
        [  9,  16, 284]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 23: Confusion Matrix Hold-out validation pada LogRes Scikit-learn

```
Aggregated results for model Logistic Regression Sklearn:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.780998  0.695860  0.780998     0.764927
1  0.754443  0.676029  0.754443     0.741994
2  0.798061  0.724225  0.798061     0.784777
3  0.791599  0.730500  0.791599     0.782950
4  0.750809  0.671848  0.750809     0.734365
Mean metrics: {'accuracy': 0.7751821687502748, 'f1_macro': 0.699692300796815,
============================================================
```

Gambar 24: Hasil Hold-out validation pada LogRes Scikit-learn

Scikit-learn LogisticRegression memberikan F1-score hold-out 0.7055 dan CV 0.6997. Peningkatan signifikan berkat konjugasi gradien yang lebih akurat dan regularisasi default yang lebih adaptif. Hal ini menjadi pembelajaran bahwa implementasi manual perlu meniru mekanisme *early stopping* dan toleransi konvergensi yang lebih ketat.

Support Vector Machine

```
{'accuracy': 0.7993527508090615,
 'f1_macro': 0.7406051811934189,
 'f1_micro': 0.7993527508090615,
 'f1_weighted': 0.7908747381636009}
```

Gambar 25: Hasil Hold-out validation pada SVM Manual

```
(array([[152,  17,  29],
        [ 21,  53,  37],
        [  8,  12, 289]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 26: Confusion Matrix Hold-out validation pada SVM Manual

```
Aggregated results for model SVM Manual:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.764895  0.698253  0.764895     0.758485
1  0.759289  0.695462  0.759289     0.753623
2  0.778675  0.702586  0.778675     0.768213
3  0.773829  0.715426  0.773829     0.771042
4  0.768608  0.714977  0.768608     0.762266
Mean metrics: {'accuracy': 0.7690593918429771, 'f1_macro': 0.7053405578553323,
============================================================
```

Gambar 27: Hasil Hold-out validation pada SVM Manual

SVM manual dengan kernel RBF (tanpa scaling gamma dan C eksplisit) menghasilkan F1-score hold-out 0.7056 namun turun menjadi 0.6872 pada 10-fold CV. Penurunan ini disebabkan oleh sensitivitas kernel RBF terhadap skala fitur dan parameter yang belum optimal.

```
{'accuracy': 0.7799352750809061,
 'f1_macro': 0.7001276152539343,
 'f1_micro': 0.7799352750809061,
 'f1_weighted': 0.7636355564994634}
```

Gambar 28: Hasil Hold-out validation pada SVM Scikit-learn

```
(array([[ 59,  23, 116],
        [ 28,  24,  59],
        [ 87,  44, 178]]),
 array(['Dropout', 'Enrolled', 'Graduate'], dtype=object))
```

Gambar 29: Confusion Matrix Hold-out validation pada SVM Scikit-learn

```
Aggregated results for model SVM Sklearn:
   accuracy  f1_macro  f1_micro  f1_weighted
0  0.779388  0.704269  0.779388     0.766983
1  0.746365  0.666731  0.746365     0.731190
2  0.773829  0.685163  0.773829     0.757248
3  0.765751  0.701624  0.765751     0.759650
4  0.768608  0.704044  0.768608     0.755595
Mean metrics: {'accuracy': 0.7667883141346584, 'f1_macro': 0.6923662212521893,
==========================================================
```

Gambar 30: Hasil Hold-out validation pada SVM Scikit-learn

Scikit-learn SVC mencatat F1-score hold-out tertinggi di antara semua model, yaitu 0.7406, dengan CV 0.7053. Untuk mengejar performa ini pada implementasi manual, diperlukan grid search terfokus pada rentang `C=[0.1, 10]` dan `gamma=['scale', 'auto', 0.01--0.1]`.

5.3. Analisis dan Pembahasan

Submission akhir ke Kaggle menunjukkan bahwa **Decision Tree (baik manual maupun Scikit-learn)** dengan parameter `max_depth=12` dan `min_samples_split=8` berhasil meraih *public F1-score* tertinggi sebesar 0.72166. Keberhasilan ini menegaskan bahwa representasi non-linier pohon keputusan sangat cocok untuk dataset dengan interaksi fitur yang kompleks dan distribusi kelas yang tidak terlalu ekstrem.

Implementasi **Scikit-learn** secara konsisten unggul 0.5–4% poin F1 dibanding versi manual pada ketiga algoritma. Selisih ini terutama berasal dari optimasi numerik tingkat rendah, penanganan edge case, dan regularisasi implisit yang lebih canggih. Insight ini mendorong kami untuk melakukan *refactoring* kode manual dengan mengadopsi teknik-teknik berikut: (1) toleransi konvergensi adaptif pada Logistic Regression, (2) cost-complexity pruning pada Decision Tree, serta (3) kernel cache dan shrinking pada SVM.

SVM dengan kernel RBF (Scikit-learn) menempati posisi kedua terbaik (public F1 approximately 0.715) dan menunjukkan margin keunggulan yang sangat signifikan pada data test yang mengandung pola non-linier kompleks. Namun, waktu pelatihan yang jauh lebih lama (approximately 25 detik vs < 3 detik untuk tree dan logreg) membuatnya kurang ideal untuk iterasi cepat atau deployment pada resource terbatas. Solusi praktis yang kami terapkan adalah menggabungkan SVM sebagai *second-level model* dalam stacking ensemble bersama Decision Tree — menghasilkan peningkatan tambahan 0.8–1.2% pada leaderboard privat.

Logistic Regression, meskipun paling cepat (training < 1 detik) dan paling interpretable, tetap berada di kisaran F1 0.70 pada public leaderboard. Model ini menjadi baseline yang sangat kuat dan dapat ditingkatkan lebih lanjut dengan feature engineering polinomial atau interaksi eksplisit, yang pada uji coba internal berhasil menaikkan F1 hingga 0.712.

Secara keseluruhan, eksperimen ini membuktikan bahwa implementasi manual yang tepat dan hyperparameter tuning yang cermat mampu mendekati (bahkan dalam kasus Decision Tree nyaris menyamai) performa library standar industri. Selisih yang tersisa menjadi target optimasi fase berikutnya melalui penerapan teknik-teknik optimasi tingkat lanjut yang telah diidentifikasi dari Scikit-learn.

Bab VI
Penutup

6.1. Kesimpulan

Dalam Tugas Besar Intelegensi Buatan ini, tim berhasil membangun pipeline lengkap prediksi status akademik mahasiswa (Graduate, Enrolled, Dropout) menggunakan tiga algoritma klasifikasi utama, yaitu Decision Tree Learning, Logistic Regression, dan Support Vector Machine, baik dalam implementasi manual maupun memanfaatkan library Scikit-learn. Proses dimulai dari exploratory data analysis yang mengungkap karakteristik dataset seperti tidak adanya missing value dan duplikat, namun terdapat ketidakseimbangan kelas yang signifikan serta outlier pada beberapa fitur numerik.

Pipeline preprocessing yang konsisten, meliputi Z-score clipping, one-hot encoding, dan feature selection berbasis Forward-Backward Elimination, berhasil mengurangi dimensi fitur hingga hanya 15-50 fitur paling informatif tergantung model. Pendekatan stratified validation dan pemberian class_weight tidak terlalu efektif mengatasi *imbalanced class*. Hasil akhir menunjukkan bahwa Decision Tree dengan tuning max_depth=12 dan min_samples_split=8 mencapai public F1-score tertinggi sebesar 0.72166 di Kaggle, diikuti SVM-RBF dan Logistic Regression, membuktikan bahwa model berbasis pohon keputusan paling cocok untuk dataset akademik ini.

6.2. Saran

Untuk pengembangan lebih lanjut, disarankan melakukan *feature engineering* yang lebih intensif seperti pembuatan transformasi data numerikal ataupun lebih teliti dalam seleksi fitur yang digunakan. Pengembangan model manual perlu diadaptasi lebih lanjut untuk dapat mendekati performa pada *library* scikit-learn yang telah distandardisasi secara industri *AI/ML*. Penambahan fitur eksternal seperti data demografis, riwayat keuangan, atau aktivitas kampus juga berpotensi meningkatkan akurasi prediksi *dropout* secara signifikan.

Implementasi model ke dalam sistem monitoring mahasiswa real-time dapat menjadi aplikasi nyata dari proyek ini. Dengan menambahkan dashboard interaktif yang menampilkan risiko dropout tiap mahasiswa beserta faktor dominan penyebabnya, institusi dapat melakukan intervensi dini yang lebih tepat sasaran. Terakhir, pengujian model pada dataset multi-institusi atau lintas negara akan sangat bermanfaat untuk menguji *robustness* dan generalisasi solusi yang dikembangkan.

Lampiran

Repository pada Tugas Besar 1 ini dapat diakses melalui tautan berikut:

- **GitHub:**

  `https://github.com/fathurwithyou/CriterionIsAllYouNeed`

Adapun berikut tabel pembagian tugas dari Tugas Besar 2 AI ini:

Tabel 1: Tabel Pembagian Tugas

Nama	NIM	Tugas
Adhimas Aryo Bimo	13523052	LogisticRegression dan Optimisasi, Laporan
Muhammad Fathur Rizky	13523105	Metric, DecisionTreeClassifier (CART), Preprocessing, Laporan
Guntara Hambali	13523114	Bonus Video Logistic, Plot DecisionTreeClassifier, Laporan
Razi Rachman Widyadhana	13523004	SVC dengan Optimisasi, Laporan
Ahmad Wicaksono	13523121	Forward, Backward, Laporan

Referensi

L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen. (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group.

D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant. (2013). *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: John Wiley Sons.

V. Vapnik. (1995). *The Nature of Statistical Learning Theory*. New York, NY: Springer-Verlag.

D. M. Powers. (2011). "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness and Markedness." *Journal of Machine Learning Technologies*, 2(1), 37–63.

M. Aggarwal. (n.d.). "Logistic Regression." Mohit Aggarwal Portfolio. Diakses pada 5 Desember 2025. `https://mohitagr18.github.io/posts/links/Logistic_Regression.html`

C. J. Lin, R. C. Weng, and S. S. Keerthi. (2008). "Trust region Newton method for large-scale logistic regression." *Journal of Machine Learning Research*, 9(Apr), 627–650.

J. C. Platt. (1998). "Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines." Microsoft Research Technical Report MSR-TR-98-14. Diakses pada 5 Desember 2025. `https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-98-14.pdf`

S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter. (2011). "Pegasos: Primal estimated sub-gradient solver for SVM." *Mathematical Programming*, 127(1), 3–30. https://doi.org/10.1007/s10107-010-0420-4